Cracking Recruit: A Tale of Log Leaks and SQL Injection

A deep dive into how "temporary" deployment fixes lead to permanent security compromises.

> **Ethical Disclosure:** This write-up is for educational purposes only. All testing was performed in a controlled laboratory environment on the TryHackMe platform.

When I first jumped onto the **Recruit** machine, I expected a standard web challenge. What I found was a classic example of how a "temporary" fix during deployment can become a permanent security nightmare. Here is how I went from an outside observer to full administrative control.

1. The Initial Scoped-In

Every good engagement starts with Nmap. I wanted to see what doors were open before I started knocking. The scan revealed three primary entry points:

- **Port 22 (SSH):** Running OpenSSH 8.2p1.
- **Port 53 (DNS):** ISC BIND 9.16.1 was active.
- **Port 80 (HTTP):** An Apache server hosting the "Recruit" portal.

```
# Nmap scan results PORT STATE SERVICE VERSION 22/tcp open ssh OpenSSH 8.2p1 Ubuntu 53/tcp open domain ISC BIND 9.16.1 80/tcp open http Apache httpd 2.4.41
```

2. Poking Around the Web App

I fired up **Gobuster** to see what was hidden under the hood. Most of the results were standard PHP files, but two things caught my eye: a `/mail` directory that seemed out of place, and a script called `file.php` that smelled like a potential LFI (Local File Inclusion) target.

I headed over to `/mail` and hit the jackpot: a file named `mail.log`.

3. The "Human Error" Moment

Inside the logs, I found a conversation between HR and IT Support discussing the new recruitment portal deployment. One line in particular made my eyes light up:

> *"HR login credentials (username: hr) are currently stored in the application configuration file (config.php) for ease of access during the initial rollout phase."*

This is a classic "we'll fix it later" mistake. Now, I just needed a way to read that `config.php` file.

4. Exploiting the API

The `file.php` script was used to fetch candidate CVs via a URL parameter: `/file.php?cv=<URL>`. I tested it with a `file://` wrapper to see if I could pull local files.

Payload: `http://[TARGET_IP]/file.php?cv=file://config.php`

```
<?php // Application Configuration $SAPP_NAME = 'Recruit';
$HR_PASSWORD = '[redacted]'; ?>
```

It worked perfectly. I had the credentials for the HR portal.

5. Escalating to Admin

With the HR credentials, I logged in and found a candidate search dashboard. It looked like a prime spot for SQL Injection. After determining the table had 4 columns using a `UNION SELECT` attack, I dumped the `users` table.

Payload: `' UNION SELECT NULL, id, password, username FROM users-- -`

This leaked the admin credentials and granted me the **ADMIN Flag**.

Lessons Learned

This machine was a great reminder that security isn't just about software bugs; it's about **processes**.

- Never leave sensitive logs in a web-accessible directory.
- Never store plaintext credentials in your code, even "temporarily."
- Always sanitize user inputs to prevent SQLi and LFI.